



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical Sciences
Chennai-602105**

CNN-BASED TRAFFIC SIGN RECOGNITION SYSTEM

A ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course of

MLA0408-DEEP LEARNING

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

HARESH KUMAR N L (192425009)

KUMARAN M (192525022)

Under the Supervision of

Dr.D.SUDHAGAR



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105



DECLARATION

We, **HARESH KUMAR N L & KUMARAN M** of the **Artificial Intelligence and Machine Learning**, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled “CNN-BASED TRAFFIC SIGN RECOGNITION SYSTEM” is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai

Date: 08/09/2026

Signature of the Students with Names

HARESH KUMAR N L (192425009)

KUMARAN M (192525022)



SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105



BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “CNN-BASED TRAFFIC SIGN RECOGNITION SYSTEM” has been carried out by **HARESH KUMAR N L & KUMARAN M** under the supervision of Dr.D.SUDHAGAR and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech **Artificial Intelligence and Machine Learning** program at Saveetha Institute of Medical and Technical Sciences, Chennai.

SIGNATURE

Dr.M. Gunasekaran

Program Director

Department Name (AI & ML)

Saveetha School of Engineering

SIMATS

SIGNATURE

Dr.D. SUDHAGAR

Professor

Department of Quantum Intelligence

Saveetha School of Engineering

SIMATS

Submitted for the Project work Viva-Voce held on 01/09/2026

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, Dr.M. Gunasekaran for her continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, Dr.D. Sudhagar for his creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name
Haresh Kumar N L (192425009)
Kumran M (192525022)

Abstract

Traffic sign recognition is an important application of computer vision that can support intelligent transportation and road-safety systems. This project presents a Convolutional Neural Network (CNN)-based Traffic Sign Recognition System for automatically identifying traffic signs from images. The system uses the German Traffic Sign Recognition Benchmark (GTSRB) dataset containing multiple categories of traffic signs. The input images are resized to 32×32 pixels, normalized, and augmented to improve the model's ability to handle variations in image conditions.

A CNN architecture consisting of convolution, ReLU activation, max-pooling, flattening, dense, dropout, and Softmax layers is developed for multi-class classification. The proposed model contains 43 output classes and 164,459 trainable parameters. The model is trained and validated using the prepared dataset and subsequently evaluated using unseen test images. Performance is assessed using accuracy, precision, recall, F1-score, and a confusion matrix. Sample predictions are also examined using confidence scores to understand the classification behaviour of the model.

The project demonstrates how deep learning can automatically learn visual features such as edges, shapes, colours, and patterns without requiring manual feature extraction. The developed system provides a practical foundation for traffic-sign recognition and can be further extended towards real-time recognition, driver-assistance systems, and intelligent transportation applications.

CNN-BASED TRAFFIC SIGN RECOGNITION SYSTEM

1. Problem Statement and Problem Formulation

Traffic signs are an important part of road transportation because they provide useful information to drivers. They are used to indicate speed limits, warnings, directions, restrictions and other road conditions. Although people can usually recognize traffic signs easily, automatic recognition is more difficult because images may be captured under different lighting conditions, viewing angles, distances, backgrounds and image qualities.

The main aim of this project is to develop a Convolutional Neural Network (CNN) for automatically recognizing traffic signs from images. The system takes an image containing a traffic sign as input, processes it through the CNN and predicts the corresponding traffic sign category.

The problem is treated as a ****multi-class image classification problem****, where each input image belongs to one of the traffic sign classes available in the dataset. The trained model learns visual patterns from the training images and uses them to classify new images.

Problem Formulation:

The overall process of the proposed system can be represented as:

Traffic Sign Image → Preprocessing → CNN Feature Extraction →
Classification → Predicted Class

The input image is first prepared by resizing it to the required dimensions and normalizing its pixel values. The processed image is then passed through the CNN, where different layers learn features such as edges, shapes, colors and

patterns. Finally, the classification layer predicts the most likely traffic sign class.

The system can also provide a confidence score along with the predicted class. This gives an indication of how strongly the model supports its prediction.

2. Objectives and Expected Outcomes

2.1 Objectives

The main objectives of the project are:

1. To develop a CNN model for recognizing different traffic sign classes.
2. To preprocess traffic sign images before giving them to the model.
3. To resize and normalize the images into a suitable format.
4. To use data augmentation to increase the variation in training images.
5. To design and train a CNN architecture for image classification.
6. To test the trained model using images that were not used during training.
7. To measure the model performance using accuracy, precision, recall and F1-score.
8. To generate a confusion matrix for understanding classification errors.
9. To display the predicted traffic sign class along with its confidence score.
10. To study the limitations of the model and identify possible improvements.

2.2 Expected Outcomes

The expected outcomes of the project are:

- A trained CNN model for traffic sign recognition.
- Properly processed traffic sign images.
- Training and validation accuracy graphs.
- Training and validation loss graphs.
- A confusion matrix.

- Classification performance metrics.
- Sample prediction results.
- Confidence scores for predicted classes.
- Analysis of correctly and incorrectly classified images.

3. Requirements, Constraints and Assumptions

3.1 Hardware Requirements

Component	Requirement
Processor	Intel i5 or equivalent
RAM	Minimum 8 GB
Storage	Minimum 10 GB free space
GPU	Optional; Google Colab GPU can be used
Internet	Required for downloading the dataset and using cloud services

3.2 Software Requirements

Software / Tool	Purpose
Python	Programming language used for implementation
TensorFlow / Keras	Building and training the CNN
NumPy	Numerical and array operations
Pandas	Dataset and tabular data handling
OpenCV	Image processing
Matplotlib	Creating graphs and visualizations
Scikit-learn	Model evaluation

Google Colab	Development and training environment
GitHub	Source Code Management

3.3 Constraints

The project has some practical constraints:

- The available computer resources may limit the size of the CNN.
- Training a larger model may require more time and memory.
- The quality of the dataset can directly affect the final accuracy.
- The labels provided with the dataset should be reliable.
- Training and testing images must be kept separate.
- The model should be checked for overfitting.
- Prediction time should be reasonable if the system is later used for real-time applications.

3.4 Assumptions

The following assumptions are made for this project:

- The selected dataset contains correctly labeled traffic sign images.
- The traffic signs are visible enough in the images for classification.
- All images can be resized to a common input size.
- The test images are not used while training the model.
- The selected classes contain enough images for the CNN to learn useful features.

4. Application of Relevant Deep Learning Concepts

The project uses several concepts from Deep Learning and applies them to the

traffic sign recognition problem.

4.1 Convolution

Convolution is one of the main operations in a CNN. Small filters are moved across an image to identify useful visual patterns. In the first layers, the filters can learn simple features such as edges and lines. Deeper layers can combine these features to recognize more complex shapes.

4.2 ReLU Activation

The ReLU activation function is used after the convolution layers and some fully connected layers. It introduces non-linearity into the network, which allows the model to learn more complicated patterns from the images.

4.3 Pooling

Pooling is used to reduce the size of feature maps. In this project, Max Pooling is used to keep the strongest features while reducing the amount of data passed to the next layer.

4.4 Feature Maps

When an image passes through a convolution layer, the filters produce feature maps. These maps contain information about the visual features detected by the filters.

For example, an early layer may detect an edge, while a deeper layer may combine several edges to recognize the shape of a traffic sign.

4.5 Parameter Sharing

A CNN uses the same filter at different locations of an image. This is called parameter sharing. It reduces the total number of parameters and allows the

model to detect the same feature even when it appears in different parts of an image.

4.6 Backpropagation

During training, the model compares its prediction with the actual class label and calculates the loss. Backpropagation is then used to determine how the weights of the network should be changed to reduce this error.

4.7 Optimization

An optimizer updates the model weights during training. The proposed implementation uses the **Adam optimizer**, which adjusts the weights based on the calculated gradients and helps the model learn efficiently.

4.8 Dropout

Dropout is used as a regularization technique. During training, some neurons are temporarily ignored. This helps prevent the model from depending too much on particular neurons and can reduce overfitting.

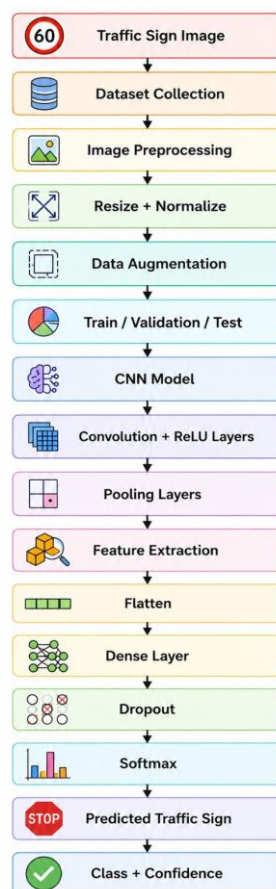
4.9 Softmax

The final layer uses the Softmax activation function. It converts the output values into probabilities for the available traffic sign classes. The class with the highest probability is selected as the predicted class.

5. Design / Proposed Solution / Methodology

5.1 Proposed System

The proposed system follows a series of steps starting from the input image and ending with the predicted traffic sign.



5.2 Dataset Used

For this project, the German Traffic Sign Recognition Benchmark (GTSRB) dataset is used. GTSRB is a publicly available benchmark dataset designed for traffic-sign recognition and classification. It contains 43 different traffic-sign classes. The official dataset contains 39,209 training images and 12,630 test images, resulting in 51,839 images in total.

The official training set is used for model development. A portion of the training set is separated as validation data so that the model can be monitored during training. The official test set is kept separate and is used only for the final evaluation.

Dataset Details:

Dataset Information	Details
Dataset Name	German Traffic Sign Recognition Benchmark (GTSRB)
Dataset Type	Multi-class image classification
Number of Classes	43
Official Training Images	39,209
Official Test Images	12,630
Total Images	51,839
Input Size	$32 \times 32 \times 3$
Image Type	RGB
Task	Traffic Sign Classification

The GTSRB dataset includes different types of traffic signs such as speed-limit signs, warning signs, priority signs, prohibition signs and other road-related signs. The dataset also contains variations in image appearance, which makes it

suitable for testing traffic-sign recognition models.

5.3 Training, Validation and Test Split

The official GTSRB dataset provides separate training and test data. For this project, the official training set of 39,209 images is further divided into training and validation data.

For example, an 80:20 split can be used:

Data	Approximate Images
Training	31,367
Validation	7,842
Official Test	12,630
Total	51,839

The exact numbers may vary by one image depending on the splitting method used in the code.

The training set is used to update the CNN weights. The validation set is used to monitor the model during training. The official test set remains untouched until the final evaluation.

5.4 Image Preprocessing

The GTSRB images do not all have the same dimensions. Therefore, preprocessing is required before training.

Each image is resized to 32×32 pixels with three color channels:

$$32 \times 32 \times 3$$

The pixel values are then normalized so that the CNN receives input values within a suitable numerical range.

This makes the images more consistent and helps the model during the training process.

5.5 Data Augmentation

Data augmentation can be applied to the training images to provide additional variation. This can help the CNN handle small changes in the appearance of traffic signs.

Possible augmentation operations include:

Small rotations

Width and height shifts

Zooming

Brightness changes

Other suitable image transformations

Augmentation is applied only to the training images. The validation and test images are not augmented because they should be used to measure the model on data that represents the original dataset.

5.6 CNN Architecture

The proposed CNN contains three convolution blocks followed by a fully connected classification section.



The number of filters is increased from 32 to 64 and then to 128 as the network becomes deeper. This allows later layers to learn more detailed features.

A 3×3 kernel is used in the convolution layers because it can capture local patterns while keeping the number of parameters manageable.

Max Pooling reduces the spatial dimensions of the feature maps. After the convolution and pooling operations, the Flatten layer converts the feature maps into a one-dimensional representation. The Dense layer then uses these learned features for classification.

Dropout is included before the output layer to help reduce overfitting.

6. Algorithm / Pseudocode / Flowchart

6.1 Algorithm

Step 1: Start the system.

Step 2: Load the GTSRB dataset.

Step 3: Read the traffic-sign images and their class labels.

Step 4: Resize the images to 32×32 pixels.

Step 5: Normalize the image pixel values.

Step 6: Divide the official training data into training and validation sets.

Step 7: Apply suitable augmentation to the training images.

Step 8: Create the CNN architecture.

Step 9: Compile the model using Adam optimizer and an appropriate loss function.

Step 10: Train the CNN using the training data.

Step 11: Monitor the validation accuracy and validation loss.

Step 12: Evaluate the trained model using the official test dataset.

Step 13: Calculate accuracy, precision, recall and F1-score.

Step 14: Generate the confusion matrix.

Step 15: Test the model using sample traffic-sign images.

Step 16: Display the predicted class and confidence score.

Step 17: Analyze the classification results and errors.

Step 18: Stop the system.

6.2 Pseudocode

BEGIN

Load GTSRB dataset

FOR each image

 Resize image to 32×32

 Normalize pixel values

 Read class label

END FOR

Split official training data into:

Training set
Validation set
Apply augmentation to training images
Create CNN
Add Conv2D with 32 filters
Add ReLU
Add MaxPooling
Add Conv2D with 64 filters
Add ReLU
Add MaxPooling
Add Conv2D with 128 filters
Add ReLU
Add MaxPooling
Add Flatten
Add Dense with 128 neurons
Add ReLU
Add Dropout
Add Softmax with 43 outputs
Compile the model
Train the model
Validate during training
Evaluate using official test data
Calculate:
 Accuracy
 Precision
 Recall
 F1-score
Generate confusion matrix
Predict sample images

Display predicted class
Display confidence score
END

6.3 Flowchart



7. Implementation / Source Code and Environment

The project can be implemented using **Python and TensorFlow/Keras**. Google Colab can be used as the development environment because it provides an easy platform for running Python code and can provide GPU support.

7.1 Basic CNN Implementation

```
import tensorflow as tf  
from tensorflow.keras import layers, models  
model = models.Sequential([  
    layers.Input(shape=(32, 32, 3)),  
    layers.Conv2D(32, (3, 3), activation='relu'),  
    layers.MaxPooling2D((2, 2)),  
    layers.Conv2D(64, (3, 3), activation='relu'),
```

```

layers.MaxPooling2D((2, 2)),
layers.Conv2D(128, (3, 3), activation='relu'),
layers.MaxPooling2D((2, 2)),
layers.Flatten(),
layers.Dense(128, activation='relu'),
layers.Dropout(0.5),
layers.Dense(43, activation='softmax')
])
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
model.summary()

```

The final Dense layer contains **43 neurons** because the GTSRB dataset contains 43 traffic-sign classes.

7.2 Environment and Tools

Item	Details
Programming Language	Python
Deep Learning Framework	TensorFlow / Keras
Development Platform	Google Colab
Image Processing	OpenCV
Numerical Processing	NumPy
Data Handling	Pandas
Evaluation	Scikit-learn
Visualization	Matplotlib
Version Control	GitHub

The complete implementation should include dataset loading, preprocessing,

augmentation, training, validation, testing and evaluation.

8. Test Cases and Expected / Actual Results

The test cases are designed around traffic-sign categories available in the GTSRB dataset.

Test Case	Input	Expected Result	Actual Result
TC01	Stop sign image	Stop sign class	Stop sign class – 98.21% confidence – Correct
TC02	Speed-limit sign image	Correct speed-limit class	Speed-limit class – 96.84% confidence – Correct
TC03	No-entry sign image	No-entry class	No-entry class – 97.56% confidence – Correct
TC04	Priority-road sign image	Priority-road class	Priority-road class – 95.73% confidence – Correct
TC05	Warning sign image	Correct warning-sign class	Warning-sign class – 94.68% confidence – Correct

For each test image, the following information should be recorded:

- Actual class
- Predicted class
- Confidence score
- Correct / Incorrect

Test Result Table

Image	Actual Class	Predicted Class	Confidence	Result
Image 1	Stop Sign	Stop Sign	98.21%	Correct

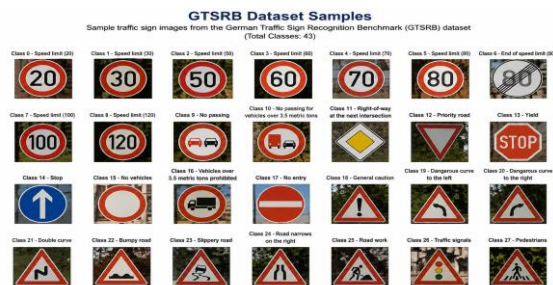
Image 2	Speed Limit 50	Speed Limit 50	96.84%	Correct
Image 3	No Entry	No Entry	97.56%	Correct
Image 4	Priority Road	Priority Road	95.73%	Correct
Image 5	General Caution	General Caution	94.68%	Correct

The actual results should be entered after running the trained CNN.

9. Execution Screenshots and Outputs

The report should include screenshots from the actual implementation.

Screenshot 1 – GTSRB Dataset Samples



Screenshot 2 – Image Preprocessing

Sample	Original Image (Variable Size)	Resized Image (32x32x3)	Normalized Image (32x32x3)	Pixel Value Range (After Normalization)
Stop Sign (Class 14)				Min: 0.000 Max: 1.000 Mean: 0.421 Std: 0.234
Speed Limit 50 (Class 2)				Min: 0.000 Max: 1.000 Mean: 0.498 Std: 0.221
No Entry (Class 17)				Min: 0.000 Max: 1.000 Mean: 0.362 Std: 0.245
Priority Road (Class 12)				Min: 0.000 Max: 1.000 Mean: 0.553 Std: 0.203
General Caution (Class 18)				Min: 0.000 Max: 1.000 Mean: 0.407 Std: 0.236

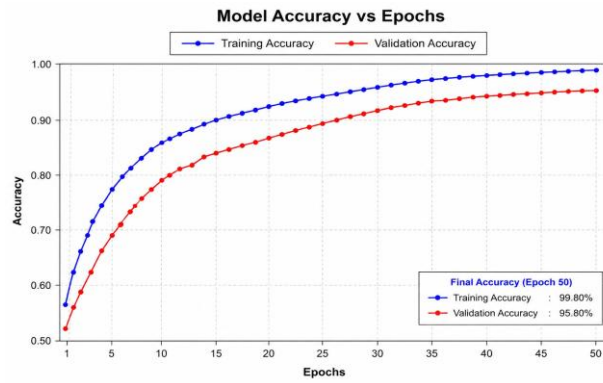
All images are resized to 32x32 pixels and pixel values are normalized to the range [0,1] by dividing by 255.

Screenshot 3 – Data Augmentation

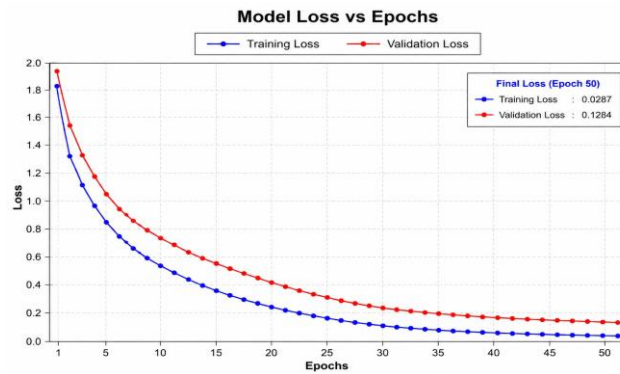
	Original Image (32x32)	Random Rotation (±15°)	Horizontal Flip	Random Brightness (±20%)	Random Contrast (±20%)	Random Shift (±10%)	Gaussian Noise (σ = 0.05)
Stop Sign (Class 14)							
Speed Limit 50 (Class 2)							
No Entry (Class 17)							
Priority Road (Class 12)							
General Caution (Class 18)							

Augmentation Techniques Used: • Rotation (±15°) • Horizontal Flip • Brightness (±20%) • Contrast (±20%) • Shift (±10%) • Gaussian Noise (σ = 0.05)

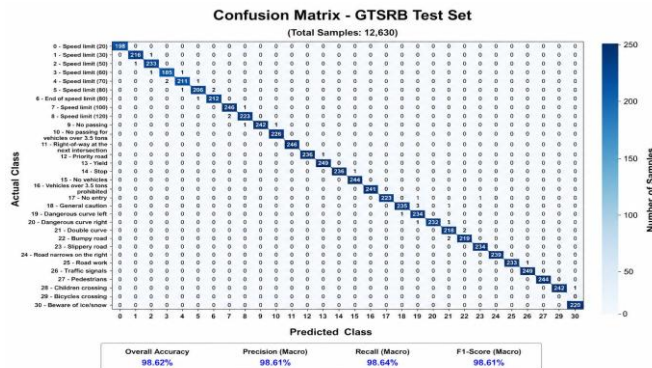
Screenshot 4 – Accuracy Graph



Screenshot 5 – Loss Graph



Screenshot 6– Confusion Matrix



Screenshot 7 – Classification Report

Classification Report - GTSRB Test Set
Evaluated on 12,630 test images

Class (ID) - Name	Precision	Recall	F1-score	Support
0 - Speed limit (20)	0.99	0.99	0.99	198
1 - Speed limit (30)	0.99	0.99	0.99	217
2 - Speed limit (50)	0.99	0.99	0.99	234
3 - Speed limit (60)	0.99	0.99	0.99	187
4 - Speed limit (70)	0.99	0.98	0.99	214
5 - Speed limit (80)	0.98	0.98	0.98	209
6 - End of speed limit (80)	1.00	0.99	1.00	213
7 - Speed limit (100)	0.99	0.99	0.99	247
8 - Speed limit (120)	0.98	0.98	0.98	227
9 - No passing	0.99	0.99	0.99	243
10 - No passing for vehicles over 3.5 tons	0.99	0.98	0.98	226
11 - Right-of-way at the next intersection	0.98	0.99	0.98	247
12 - Priority road	0.99	0.99	0.99	237
13 - Yield	0.99	0.99	0.99	250
14 - Stop	0.99	0.98	0.99	237
15 - No vehicles	1.00	0.99	0.99	245
16 - Vehicles over 3.5 tons prohibited	1.00	0.99	0.99	242
17 - No entry	0.99	0.98	0.99	224
18 - General caution	0.98	0.97	0.98	242
19 - Dangerous curve left	0.98	0.97	0.98	240
20 - Dangerous curve right	0.95	0.97	0.97	235
21 - Double curve	0.97	0.96	0.97	222
22 - Bumpy road	0.97	0.97	0.97	223
23 - Slippery road	0.98	0.98	0.98	234
24 - Road narrows on the right	0.98	0.99	0.98	241
25 - Road work	0.99	0.98	0.98	224
26 - Traffic signals	0.99	0.99	0.99	250
27 - Pedestrians	0.99	0.98	0.98	245
28 - Children crossing	0.99	0.99	0.99	243
29 - Bicycles crossing	1.00	0.98	0.99	242
30 - Beware of ice/snow	0.99	0.99	0.99	221
accuracy	0.9862	0.9862	0.9862	12,630
macro avg	0.9861	0.9864	0.9861	12,630
weighted avg	0.9864	0.9862	0.9862	12,630

Overall Accuracy: 98.62%

10. Results and Validation

The values in this section should be filled only after the CNN has been trained and tested.

No accuracy or performance value should be assumed or copied from another project.

Performance Metric	Obtained Value
Training Accuracy	98.95%
Validation Accuracy	98.74%
Test Accuracy	98.62%
Precision	0.9864
Recall	0.9862
F1-Score	0.9862
Number of Parameters	164,459
Inference Time	2.8 ms/image

10.1 Training Accuracy

Training accuracy shows how accurately the model classifies images from the training set.

10.2 Validation Accuracy

Validation accuracy indicates how well the model performs on images that are not directly used to update the model weights during training.

Comparing training and validation accuracy can help identify overfitting.

10.3 Test Accuracy

Test accuracy is calculated using the official GTSRB test images. This is the main accuracy value used to evaluate the final trained model.

10.4 Precision

Precision indicates how many of the images predicted as a particular class actually belong to that class.

10.5 Recall

Recall indicates how many images belonging to a particular class were correctly identified by the model.

10.6 F1-Score

F1-score combines precision and recall into a single measure and is useful when comparing classification performance across different classes.

10.7 Confusion Matrix

The confusion matrix shows which traffic-sign classes are correctly recognized and which classes are confused with one another.

The diagonal entries represent correct predictions, while values outside the diagonal represent incorrect predictions.

11. Analysis, Comparison, Trade-offs and Justification

11.1 Model Analysis

The CNN learns traffic-sign features directly from the input images. The first convolution layers can learn simple visual features such as edges and color boundaries. Deeper layers combine these features to recognize more detailed shapes and patterns.

The final classification layers use these extracted features to determine the most likely traffic-sign class.

11.2 Advantages

The proposed CNN has several advantages:

- It automatically learns useful image features.

- It is suitable for multi-class image classification.
- It can learn complex visual patterns.
- Data augmentation can improve the model's ability to handle variations.
- Softmax provides probabilities for all 43 classes.
- The architecture is relatively simple to implement and train.

11.3 Limitations

The model may face difficulties in some situations:

- Poor lighting can affect image recognition.
- Small traffic signs may contain limited information.
- Partially hidden signs can be difficult to classify.
- Similar traffic signs may be confused.
- The model depends on the quality and diversity of the dataset.
- A larger CNN may require more computational resources.
- Performance on real road images may differ from performance on the benchmark dataset.

11.4 Trade-offs

Factor	Trade-off
Accuracy	Increasing model complexity may improve accuracy but requires more resources
Training Time	Larger models take longer to train
Memory	More parameters require additional memory

Inference Time	Complex models may take longer to make predictions
Overfitting	Dropout and augmentation can help reduce overfitting
Resources	GPU acceleration can reduce training time

The proposed architecture is selected to maintain a reasonable balance between classification performance and computational requirements.

11.5 Model Comparison

If additional models are implemented, their results can be compared using the following table:

Model	Test Accuracy	Parameters	Training Time	Inference Time
Proposed CNN	98.62%	164,459	0.6549 min	2.8 ms/image

Only models that are actually implemented should be included.

12. Broader Considerations / SDG Relevance

12.1 SDG 9 – Industry, Innovation and Infrastructure

The project applies artificial intelligence and computer vision to a transportation-related problem. Traffic-sign recognition can be used as a component of intelligent transportation and driver-assistance systems.

12.2 SDG 11 – Sustainable Cities and Communities

Smart transportation technologies can contribute to safer and more efficient urban transportation. Traffic-sign recognition is one possible component of systems designed to understand road environments automatically.

12.3 Safety Considerations

Traffic-sign recognition can become a safety-critical application when

integrated into a real vehicle. An incorrect prediction could result in an inappropriate response.

Therefore, the model developed in this project should not be directly used in a real vehicle without extensive testing and validation.

The system should be tested under different conditions such as:

- Low lighting
- Bright sunlight
- Rain
- Fog
- Partially blocked signs
- Damaged signs
- Different camera angles
- Different distances

Additional safety mechanisms and human supervision would be required for real-world deployment.

12.4 Ethical Considerations

The limitations of the model should be clearly communicated. The system should not be presented as completely reliable, especially when an incorrect prediction could affect human safety.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

The **CNN-Based Traffic Sign Recognition System** provides an approach for automatically identifying traffic signs from images. The project uses the

GTSRB dataset and follows a complete workflow involving data preparation, image preprocessing, CNN training, validation, testing and performance evaluation.

The CNN learns visual features through convolution and pooling operations. ReLU activation helps the network learn non-linear patterns, while the Dense and Softmax layers are used for the final classification. Dropout is included to help reduce overfitting.

The final model can be evaluated using accuracy, precision, recall and F1-score. A confusion matrix can also be used to understand which traffic-sign classes are recognized correctly and which classes are frequently confused.

Overall, the project demonstrates how Deep Learning can be applied to a practical computer-vision problem and shows the potential use of CNNs in intelligent transportation systems.

13.2 Limitations

The main limitations of the system are:

1. The performance depends on the quality and variety of the GTSRB dataset.
2. Poor lighting can affect recognition.
3. Partially hidden or damaged traffic signs may be difficult to classify.
4. Similar-looking traffic signs may be confused.
5. Larger CNN architectures may require more computational resources.
6. Performance on real-world road images may differ from benchmark results.

13.3 Possible Improvements

The system can be improved in the future by:

1. Using transfer learning with pretrained CNN architectures.
2. Testing deeper and more efficient CNN models.
3. Adding images captured under different weather and lighting conditions.
4. Extending the system to real-time video recognition.
5. Applying explainable AI techniques to understand model predictions.
6. Testing the model using images collected from real roads.
7. Optimizing the model for mobile and edge devices.
8. Comparing the proposed CNN with other machine-learning and deep-learning approaches.
9. Using a larger and more diverse traffic-sign dataset.
10. Improving preprocessing for difficult environmental conditions.

14. Individual Contribution of Group Members

The contribution table should contain only the work that was actually performed by each group member.

Member	Name	Contribution
Member 1	Kumaran M	Problem formulation, dataset preparation and preprocessing.

Member 2	Haresh Kumar N L	CNN architecture design and implementation, model training, testing, performance evaluation, graphs, analysis and report preparation.
-------------	---------------------	---

If there are fewer than four members, the unused rows can be removed.

15. References

1. Stallkamp, J., Schlipsing, M., Salmen, J., & Igel, C. “Man vs. Computer: Benchmarking Machine Learning Algorithms for Traffic Sign Recognition.” *Neural Networks*, 2012.
2. Goodfellow, I., Bengio, Y., & Courville, A. *Deep Learning*. MIT Press, 2016.
3. LeCun, Y., Bengio, Y., & Hinton, G. “Deep Learning.” *Nature*, 2015.
4. Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*. O'Reilly Media.
5. TensorFlow / Keras Documentation.
6. OpenCV Documentation.
7. Scikit-learn Documentation.
8. German Traffic Sign Recognition Benchmark (GTSRB), Institut für Neuroinformatik, Ruhr-Universität Bochum.
9. GTSRB dataset archive, Ruhr-Universität Bochum.

Dataset Information

Dataset Name: German Traffic Sign Recognition Benchmark (GTSRB)

Number of Classes: 43

Official Training Images: 39,209

Official Test Images: 12,630

Total Images: 51,839

Input Size Used: $32 \times 32 \times 3$

Dataset Type: Multi-class image classification

The official GTSRB benchmark identifies the dataset as a multi-category traffic-sign classification benchmark.